Example of Full System Implementation

Stage 3: SoC Place and Route

1 Objective

In this final project, you will get an overall perspective of everything we learned in the course and a bit more, by running a full system implementation. This implementation will take a simple piece of **C** code, compile it for a **RISC-V** processor implementation, run a cycle-accurate simulation of the code upon the RTL, synthesize, place and route a system-on-chip (SoC) with the processor, and run a gate-level simulation to ensure that the final design is true to the original intent and even measure the power consumption of your program.

Specifically, this project will include the following stages:

1. Create a simple SoC including a **RISC-V** processor, **SRAM** instances, and **I/O** pads.
2. Compile an example **C** program ("Bubble Sort") for a **RISC-V** architecture implementation.
3. Run a testbench in **Incisive** (irun) that loads the binaries of the compiled **C** program into the memory of the **SoC** and executes the program.
4. Synthesize the RTL of the **SoC** into a gate-level netlist.
5. Create a floorplan for the **SoC**, including deciding on the positions of the **I/O pads** and the **SRAM** macros.
6. Continue the place and route flow to get a **DRC** clean layout that meets **timing**.
7. *Optional:* Run a gate-level simulation on the post-layout netlist to verify design correctness.
8. *Optional:* Estimate the power consumption due to gate toggling and leakage when running your **C** program.

2 Exercise Overview

This exercise will be divided into four stages, as follows:

- **Stage 1**: The RISC-V Toolchain: During this stage, you will take a program written in a high-level language, compile it, load the binaries into the processor memory, and run a logic simulation on the processor RTL.
- **Stage 2**: Synthesis: During this stage, you will run synthesis on the **SoC**. To do this, you will need to first compile **memory** macros and integrate them into the RTL.
- **Stage 3**: Place & Route: During this stage, you will floorplan your chip and run place and route to provide a post-layout block. To do this, you will need to define your I/O ring.
- ***Optional* Stage 4**: Gatelevel Simulation: During this stage, you will verify that your C program runs on the post-layout netlist. In addition, you will save toggling information to extract power consumption.

You are now at **Stage 3** of the project, which will include physical implementation (place and route) of the SoC. But first, you will have to prepare your SoC by defining your I/Os and organizing them in an I/O ring.

Copyright © Yehuda Kra, Prof. Adam Teman, 2023

2.1 Top-level and I/O Pads

The top-level of our SoC is defined in the file `sourcecode/rtl/soc/lp_riscv_top.v` and the top-level module is named `lp_riscv_top`, accordingly. The goal of the top-level module is to wrap together the SoC top module (`lp_riscv`) with the interface to the outside world – the **I/O cells** and **bonding pads**. The top-level has:

- 8 single bit input ports (such as `PAD_CLK` and `PAD_RST_N`)
- 3 busses (such as `PAD_IRAM_PROG_BYTE[7:0]`)
- Single output bit (`PAD_DONE_FLAG`)

Each of these ports must connect from the outside world to an **I/O cell** and from there connect to the `lp_riscv` SoC. Pay attention that the interface ports to/from the external world are capitalized and start with a `PAD_` prefix.

To make the top-level file more concise, the actual instantiation of the **I/O cells** is done in a separate file, `sourcecode/rtl/soc/ioring.v`. This file includes the instantiation of actual **I/O cells** (hard macros) from the technology-specific **I/O cell library** – in our case, provided by TSMC. The **I/O library** is installed on the server, and provides all the necessary views, such as **Verilog** files for *simulation* and `.lib` files for *timing*.

Open up the `ioring.v` file and take a look at how the **I/O cell** instantiation is done. Note that the file also includes instantiation of **non-functional I/O cells**, such as Power/ground, placeholder I/O cells and corner cells.

So what is left to do inside the `lp_riscv_top.v` file? Actually, it just has wires that connect:

- Top-level input ports to "`PAD`" pin on **input-configured I/O cells**
- Wires that connect "`C`" pin of **input-configured I/O cells** to `lp_riscv` input ports.
- Wires that connect `lp_riscv` output ports to "`I`" pin of **output-configured I/O cells**.
- "`PAD`" pin of **output-configured I/O cells** to Top-level output ports.

This is visually shown in Figure 1. From **Stage 2**, we have the `riscv_core` connected to instruction and data memory banks. Now we connect the inputs and outputs from `riscv_core` to the actual pads (in the illustration we show only one input and one output pad) through I/O cells. The **input** is internally connected through the I/O cell's `C` pin and the **output** is connected through the `I` pin.

Copyright © Yehuda Kra, Prof. Adam Teman, 2023

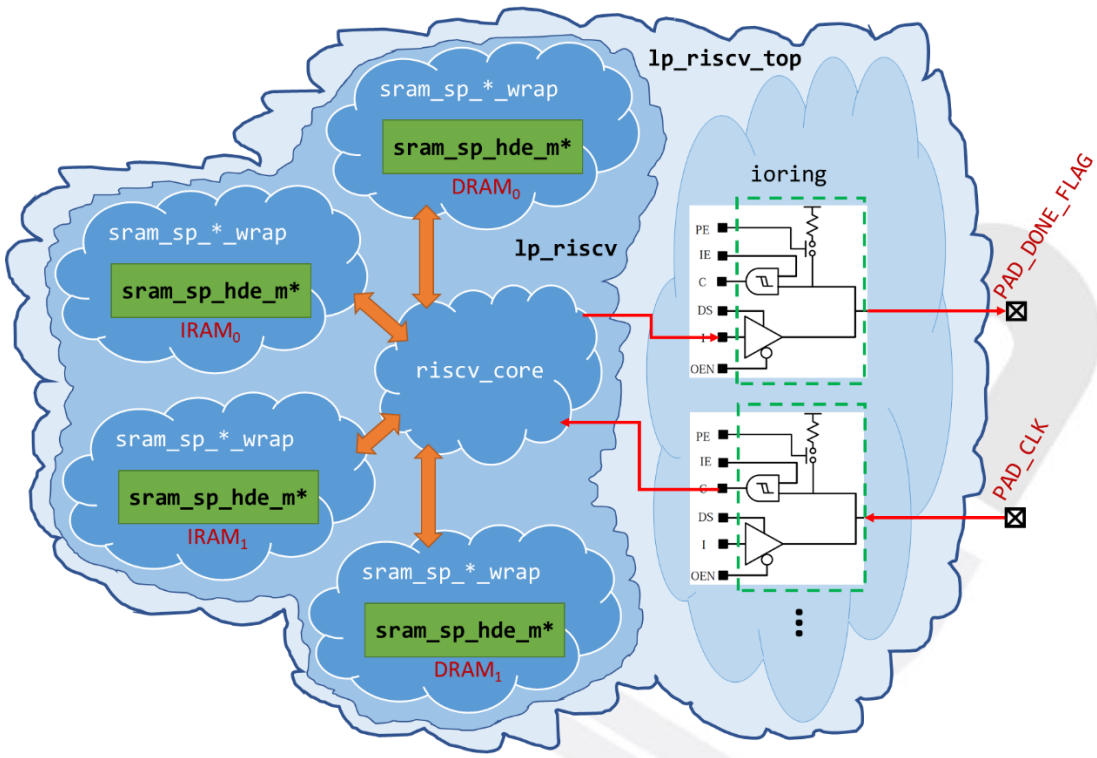

Figure 1: SoC Hierarchy

2.2 I/O Generation & Automation

Design of the I/O ring is a very important stage in designing a chip. The pads have to be placed so that they provide the best location for the connection on the board but also so that they fit the on-chip floorplan well. In addition, there are constraints for building your I/O ring, such as providing a sufficient number of `VDD`/`VDDIO`/`VSS`/`VSSIO` sets (to accommodate simultaneous switching outputs and reduce IR Drop). That being said, there are a limited number of I/Os and therefore, the I/O ring planning is commonly done by hand without any type of automatic optimization algorithm. In fact, the basic way is to manually write the structural (gate level) **Verilog** of the I/O ring file (`ioring.v`) and provide the locations for placing each of the I/Os either with the floorplanning GUI or with a simple file that states their locations (**Innovus** supports a proprietary syntax for such an I/O floorplanning file).

When you load your design into Innovus (with the `init_design` command), the tool will bind the LEFs of the instantiated I/Os and spread them around the periphery of a square floorplan. This is *not* the I/O ring you want to compose – it is just a default starting point that the tool provides. Furthermore, the size of the chip is based on the number of I/Os (for a pad limited design) or a default utilization target (for a core-limited design). But in reality, the size of your chip will be determined by additional factors, such as in the case of this homework, where we will define the floorplan size for you in Section 3. Therefore, you will run a floorplan definition command that will change the size and aspect ratio of your floorplan. The tool will rearrange your I/Os to fit the new floorplan, but this, of course, is not the I/O ring that you want. For example, you will want to distribute the power supply I/Os on all sides of the chip to robustly feed the power grid. You will also probably want to provide the Clock and Reset inputs where they can best feed the entire design, for example, at the center of one of the long sides of the chip.

Copyright © Yehuda Kra, Prof. Adam Teman, 2023

So what should we do?

Well, there can be different approaches for building the I/O ring and locating them where you want them, such as:

- Manually moving the I/Os in the GUI.
- Writing a script to place the I/Os.
- Utilizing the **IO File** format.

You can choose any of the above approaches or a combination of them. For example, start by rearranging your I/Os by moving them in the GUI to get the desired order and allocation to chip side. Then save the resulting **IO File**, by choosing FILE⊙EXPORT⊙IO. Finally, edit the resulting **IO File** to distribute the IOs evenly along the chip sides and then load it back into **Innovus**.

Note that the provided script loads an IO File from `../inputs/ioring.io`. If you choose to use an **IO File**, make sure that you name it accordingly or edit the script.

3 Project Stage 3 Instructions

During this stage, we will:

1. Define the floorplan using the I/O Generation Framework.
2. Load our design into Innovus.
3. Run Place and Route on the SoC.

3.1 Exercise Parameters

Let us define the 9-digit ID number of each student as ***ABCDEFGHI***. With this definition, find the value of ***Width*** and ***Height*** in the table below:

C	***Width***	***D***	***Height***
0	900	0	900
1	950	1	950
2	1000	2	1000
3	1050	3	1050
4	1100	4	1100
5	1150	5	1150
6	1200	6	1200
7	1250	7	1250
8	1300	8	1300
9	1350	9	1350

Copyright © Yehuda Kra, Prof. Adam Teman, 2023

3.2 Clone the repo and create a branch

```
tsmc65 && cd DVD
git clone git@github.com:DVD2026/hw6.git
cd hw6
git checkout v1.0 -b <ID>
./setup.sh
```

3.3 Scripts for Running Innovus

As usual, we have provided you with a very mature template for your design implementation, and you only need to adjust a handful of parameters in the script to carry out a full flow. In addition to files you used in Stage 2, these include:

- `../scripts/innovus.tcl` – the main implementation script.
- `../inputs/<design>.floorplan.defines` – Some specific parameters and definitions for building your floorplan.
- `../inputs/<design>.mmmc` – the MMMC file for running this process.
- `../inputs/<design>.ccopt` – the clock tree spec for running this process.

The majority of the commands within the `innovus.tcl` script are generic and can be used without modification. However, the entire **floorplanning** stage needs to be tweaked (by you!) to arrive at an efficient, DRC-clean, design.

Note that you also need to make sure your SRAMs (from Stage 2) are properly defined in the `../libraries` script, including **LEFs** and **LIBs**.

3.4 Load your Design into Innovus

Run Innovus with the -stylus option:

```
%> innovus -stylus
```

Then load the commands from the innovus.tcl file until finally reaching the `init_design` command. For `init_design` to complete successfully, you will have to make sure your libraries (SRAMs) are properly defined, your **SDC** is properly defined, and you don't have any other "surprises" (…the scripts are pretty well tailored for this tasks, so as long as you don't provide "Garbage In", you shouldn't be getting "Garbage Out"…).

Make sure you don't have any **Errors** during `init_design` and go over the **Warnings** to ensure that there is nothing critical that won't allow you to continue.

3.5 Floorplan your Design

- Set the width and height of your chip in the script according to the ***Width*** and ***Height*** parameters defined above.
- Arrange your IOs the best you can in one (or a combination) of the methods described above.
- Arrange your SRAMs within the floorplan (note that this may affect your IO definition!). Create placement halos and rings around your SRAMs. Connect your SRAM power pins to their rings.

Copyright © Yehuda Kra, Prof. Adam Teman, 2023

- Distribute well taps and end caps according to design rules.
- Create core rings and connect them to the power supply pads and follow pins.
- Create a robust power grid (stripes).
- Check DRC and make sure that everything connects cleanly and nicely.

3.6 Place and Route Your design

- Now continue through the place and route flow.
- Adjust the Clock Spec (`../inputs/<design>.ccopt`) to adhere to your design and inspect the clock tree after CTS.
- Optimize to fix all DRCs and all HOLD violations with the highest frequency you can reach.

3.7 Submission

Please add, commit and push your files that should include:

- Your final post-layout design (under the `exports` folder).
- Your critical path timing report for both SETUP and HOLD (under the `reports` folder).
- Your `check_drc` and `check_connectivity` reports (under the `reports` folder).
- A file called: `reports/my_pnr.txt` which includes:
  - First line: **Your ID Number**
  - Second line: **Width** x **Height**
  - Third line: Smallest **clock period** your design was able to meet.
  - Fourth Line: **Slack** of your worst HOLD path (must be non-negative!)
  - Fifth line: **Startpoint** of your worst HOLD path
  - Sixth line: **Endpoint** of your worst HOLD path

For example, your `my_pnr.txt` file should look something like this:

```
012345678
Floorplan:          1000 x 1000
Period:             1.500 ns
Hold Slack:         0 ps
Startpoint (Hold):  XXX/XXX/XXX_reg/CP
Endpoint (Hold):    XXX/XXX/XXX_reg/D
```


Copyright © Yehuda Kra, Prof. Adam Teman, 2023